Research note: why LLD practice is hard to evaluate

crit · LLD Practice Platform
github.com/Sumit99589/crit-lld

The learner problem

Practising low-level design is easy to start: pick “Parking Lot”, sketch some classes, maybe write code. The hard part is knowing whether the result is any good.

1. **There is no single right answer.** Two strong elevator designs can split responsibilities very differently. Comparing your design with one published solution can't tell “different” from “wrong”.
2. **Feedback is either a reference solution or a score.** A reference shows *a* design, not what is weak about *yours*. “7/10” doesn't say which class to change.
3. **Mistakes repeat silently.** The same god “Manager” class, missing failure handling, or empty abstraction shows up on every problem, and nothing tells the learner it's a pattern.

> Learners say this themselves. A widely viewed LeetCode Discuss post is titled *“Parking Lot LLD | Is this approach enough for interviews? | Feedback appreciated”* (link). On the most popular LLD repository, learners ask for test scenarios so they can check their own solutions (#188) and say that without class diagrams “it's hard to understand the connectivity between the classes” (#141).

What exists today

Eight resources, each checked on its own pages in October 2026. Some product internals sit behind a login, as noted.

Tool	How you practise	Submission	Feedback	Learning loop
AlgoMaster AI LLD interviews	Timed AI mock: requirements → entities → classes → code	Text or voice, then code	Each stage scored /10 by AI against “our reference solution”	Sessions saved; no weakness tracking; premium
awesome-low-level-design (27k★)	Read problem, read solution	None	Reference Java code	Manual checklist
Codemia OOD	~70 problems with a structured text template	Template + diagram tool	“AI Design Review” advertised; public submissions	Streaks, flashcards
Hello Interview	Breakdowns; 8 guided practice problems	Step-by-step answers	“Instant AI feedback”; LLD rubric not published; human mocks ended May 2026	Per-question history
Educative “Grokking the LLD Interview”	Case studies with UML, code, 19 AI mocks	Mock answers	AI mock; grading not public	Unknown
DesignGurus “Grokking OOD”	Lessons, 250+ diagrams, quizzes	None	Reference solutions	Quiz scores
GeeksforGeeks articles	Read	None	None (comments)	None
TUM Athena / Apollon (ICSE-SEET 2026)	University modelling exercises	UML → text	LLM feedback against rubric-style instructions, reviewed by human graders	Course-level

Gaps

1. **Feedback is anchored to one canonical answer.** Even the AI-graded tool compares with a reference solution, so a valid but *different* design looks wrong.
2. **Reference answers are unreliable.** The most popular repo has open correctness issues in its own solutions (#184: parking breaks un-parking; #73: logger “not thread safe as asked”).
3. **AI feedback is opaque.** No LLD rubric is published, and scores aren't tied to specific classes. The TUM study found LLM grades are generous to weak work and harsh to strong work.

4. **History is a log, not a coach.** Attempts are saved, but no tool I could verify surfaces recurring weaknesses across problems.
5. **Requirement change is rarely tested**, even though extensibility is the point of LLD. Practice stops at the first design.

Product direction

crit focuses on one loop: choose a problem → design → submit → get a crit → revise → see what you fixed.

- **A structured design canvas** instead of free text or code: assumptions, classes with one-sentence responsibilities, typed relationships, a requirement → class map, one key flow, decisions, and a response to a change. The structure enables deterministic checks, and the canvas draws a live UML diagram (answering #141).
- **Grading against a published rubric, not a reference solution:** six criteria with four described levels. Every judgement must quote the learner's own design, and the quotes are machine-checked.
- **Facts before opinions:** deterministic checks (unmapped requirements, ownership cycles, orphan classes, an unanswered change) run first and cap the AI where it disagrees, countering the generosity bias.
- **A requirement change in every problem**, so extensibility is actually exercised.
- **Revisions are diffed deterministically** (“fixed: unconnected DisplayBoard”), and recurring weaknesses are shown on a progress page.
- **It works without AI:** if the model is down or over budget, the learner still gets complete rule-based feedback.

Out of scope for the MVP: accounts, a problem editor, code execution, diagram input, and human review. The design leaves a place for each.